
CS 301

High-Performance Computing

Assignment 04 - Performance Analysis of Particle Interpolation and Mover Operation

Diya Patel (202301216)
Yesha Joshi (202301462)

March, 2026

Contents

1	Introduction	3
2	Hardware Details	3
2.1	Hardware Details for LAB207 PCs	3
2.2	Hardware Details for HPC Cluster	4
3	Algorithm Overview	5
3.1	Interpolation	5
3.2	Mover Operation	5
3.3	Algorithm Flow Diagram	6
4	Experimental Setup	6
4.1	Grid Configurations	6
5	Experiment 01: Scaling with Number of Particles	7
5.1	Results	7
5.2	Observations	7
6	Experiment 02: Consistency Across Configurations	8
6.1	Results	8
6.2	Observations	8
7	Experiment 03: Mover Operation and OpenMP Parallelization	9
7.1	Iteration-wise Performance	9
7.2	Serial vs Parallel Mover	10
7.3	Speedup	10
8	Questions and Analysis	11
8.1	Performance Differences Between Lab PC and HPC Cluster	11
8.2	Speedup from Parallelization	11
8.3	Parallelizing the Interpolation Function	11
9	Conclusion	11

1 Introduction

Particle-based simulations are widely used in computational physics and engineering applications where discrete particles interact with a structured grid. A common operation in such simulations is the *particle-to-mesh interpolation*, where values from scattered particle positions are distributed onto nearby grid nodes.

The performance of this operation is important because it is executed repeatedly in large-scale simulations involving millions or billions of particles. The computational cost of interpolation depends primarily on the number of particles and the efficiency of memory access patterns.

In this assignment we analyze the performance behavior of a particle interpolation algorithm and extend it by introducing a particle displacement phase called the *Mover* operation. The mover phase updates particle positions randomly within the simulation domain.

Three experiments are performed:

- Scaling behavior with increasing particle count
- Consistency of interpolation performance across grid resolutions
- Performance improvement from OpenMP parallelization of the mover operation

All experiments are executed on both a Lab PC and an HPC cluster to study the impact of hardware architecture and system characteristics on execution time.

2 Hardware Details

2.1 Hardware Details for LAB207 PCs

- Architecture: x86_64
- CPU op-mode(s): 32-bit, 64-bit
- Byte Order: Little Endian
- CPU(s): 12
- On-line CPU(s) list: 0-11
- Thread(s) per core: 2
- Core(s) per socket: 6
- Socket(s): 1
- NUMA node(s): 1
- Vendor ID: GenuineIntel
- CPU family: 6
- Model: 151

- Model name: 12th Gen Intel(R) Core(TM) i5-12500
- Stepping: 5
- CPU max MHz: 4600.0000
- CPU min MHz: 800.0000
- BogoMIPS: 5990.40
- Virtualization: VT-x
- L1d cache: 288K
- L1i cache: 192K
- L2 cache: 7.5M
- L3 cache: 18M
- NUMA node0 CPU(s): 0-11
- Flags: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm constant_tsc arch_perfmon pebs bts rep_good nopl xtopology nonstop_tsc aperfmperf eagerfpu pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3 fma cx16 xtpr pdcm pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand lahf_lm abm epb invpcid_single tpr_shadow vnmi flexpriority ept vpid fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid xsaveopt dtherm ida arat pln pts

2.2 Hardware Details for HPC Cluster

- Architecture: x86_64
- CPU op-mode(s): 32-bit, 64-bit
- Byte Order: Little Endian
- CPU(s): 16
- On-line CPU(s) list: 0-15
- Thread(s) per core: 1
- Core(s) per socket: 8
- Socket(s): 2
- NUMA node(s): 2
- Vendor ID: GenuineIntel
- CPU family: 6

- Model: 63
- Model name: Intel(R) Xeon(R) CPU E5-2640 v3 @ 2.60GHz
- Stepping: 2
- CPU MHz: 1288.726
- BogoMIPS: 5204.73
- Virtualization: VT-x
- L1d cache: 32K
- L1i cache: 32K
- L2 cache: 256K
- L3 cache: 20480K
- NUMA node0 CPU(s): 0-7
- NUMA node1 CPU(s): 8-15

3 Algorithm Overview

3.1 Interpolation

The interpolation algorithm distributes the contribution of each particle to the four surrounding grid nodes using bilinear weighting. For each particle:

1. The grid cell containing the particle is identified.
2. Local interpolation weights are computed.
3. Contributions are distributed to four neighboring grid points.

Since each particle performs a fixed number of arithmetic operations, the computational complexity of the algorithm is proportional to the number of particles.

3.2 Mover Operation

After interpolation, particles are randomly displaced within the simulation domain. The displacement limits are defined as

$$\Delta x = \frac{1}{N_x}, \quad \Delta y = \frac{1}{N_y}$$

If a displacement moves a particle outside the domain, a new displacement is generated until the particle remains within the boundaries.

3.3 Algorithm Flow Diagram

The workflow of the implemented algorithm is illustrated below. The simulation begins by initializing particle positions within the unit domain. For each iteration, interpolation distributes particle contributions to the grid. After interpolation, the mover operation randomly displaces particles while ensuring they remain within the domain boundaries.

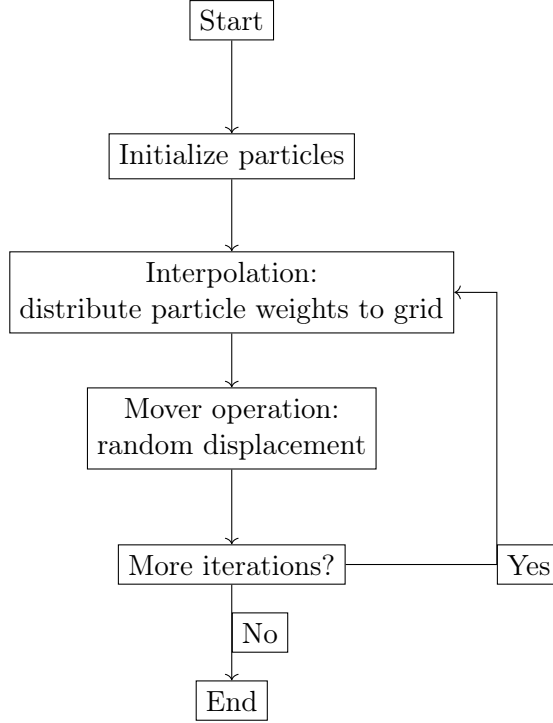


Figure 1: Schematic workflow of the particle interpolation and mover algorithm.

4 Experimental Setup

All particles and grid points lie within a 1×1 domain.

4.1 Grid Configurations

Problem Index	N_x	N_y
1	250	100
2	500	200
3	1000	400

Table 1: Grid configurations used in the experiments

Particle counts used in the experiments are

$$10^2, 10^4, 10^6, 10^8, 10^9$$

Each experiment runs for 10 iterations.

5 Experiment 01: Scaling with Number of Particles

This experiment studies how interpolation time scales as the number of particles increases. For each particle count:

- Particles are initialized
- Interpolation is executed
- Only interpolation time is accumulated

5.1 Results

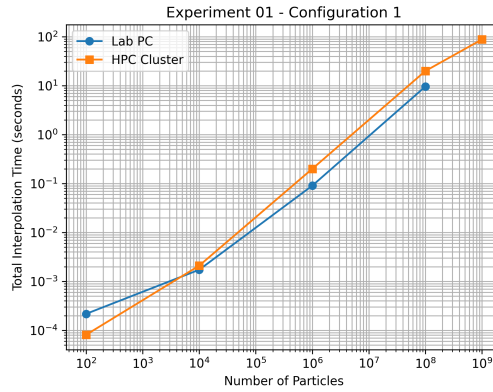


Figure 2: Log-log plot of interpolation time vs number of particles for configuration 1.

5.2 Observations

The log-log plots show an approximately linear relationship between execution time and particle count. This indicates that the interpolation algorithm scales linearly with the number of particles.

Each particle contributes to four grid nodes using a fixed number of operations, so the computational complexity of the algorithm is approximately $O(N)$.

The Lab PC generally shows lower execution time compared to the HPC cluster, which may be due to differences in processor frequency, system load, and shared resource usage on the cluster.

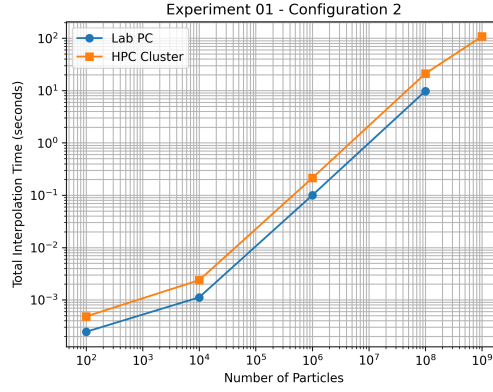


Figure 3: Log-log plot for configuration 2.

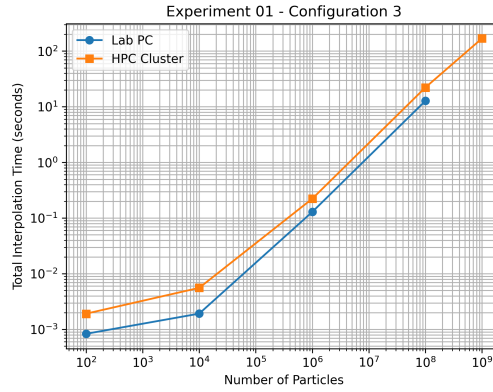


Figure 4: Log-log plot for configuration 3.

6 Experiment 02: Consistency Across Configurations

In this experiment the number of particles is fixed at 10^8 . Interpolation is executed repeatedly for different grid resolutions.

6.1 Results

6.2 Observations

The interpolation time remains relatively similar across different grid configurations. This behavior occurs because the algorithm iterates over particles rather than grid cells. Therefore, the number of particles dominates the computational cost.

A slight increase in execution time for larger grids is observed due to increased memory usage and cache effects.

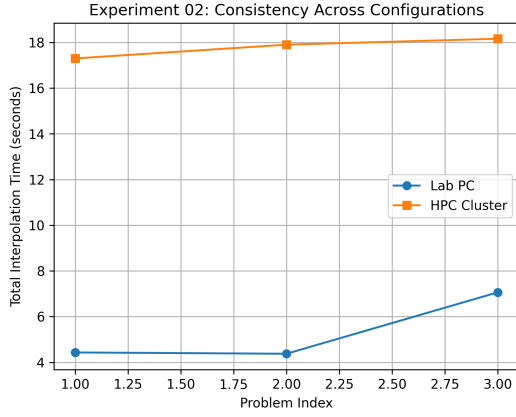


Figure 5: Total interpolation time vs problem index.

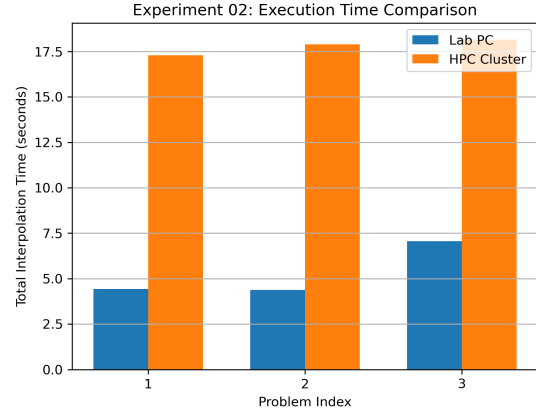


Figure 6: Bar plot of execution time vs problem index.

7 Experiment 03: Mover Operation and OpenMP Parallelization

The following parameters are used:

- $N_x = 1000$
- $N_y = 400$
- Particles = 14 million
- Threads = 4

7.1 Iteration-wise Performance

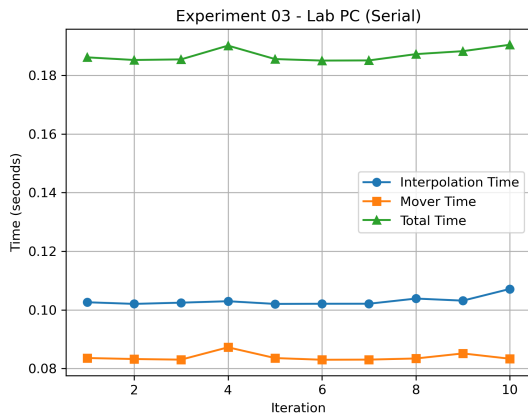


Figure 7: Interpolation time, mover time and total time per iteration on the Lab PC.

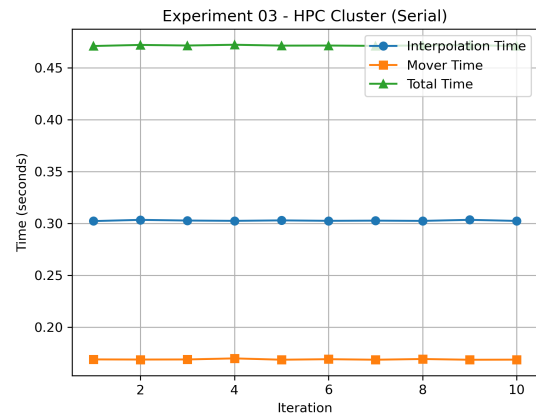


Figure 8: Interpolation time, mover time and total time per iteration on the HPC cluster.

7.2 Serial vs Parallel Mover

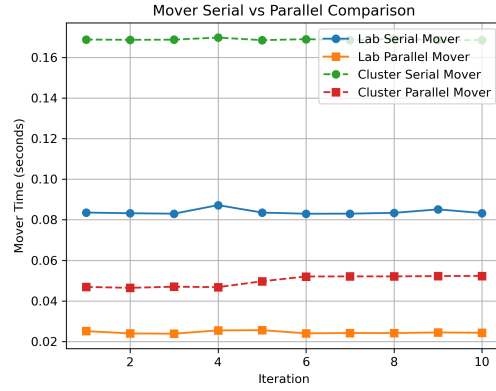


Figure 9: Comparison of serial and parallel mover execution times.

7.3 Speedup

Speedup is defined as

$$S = \frac{T_{serial}}{T_{parallel}}$$

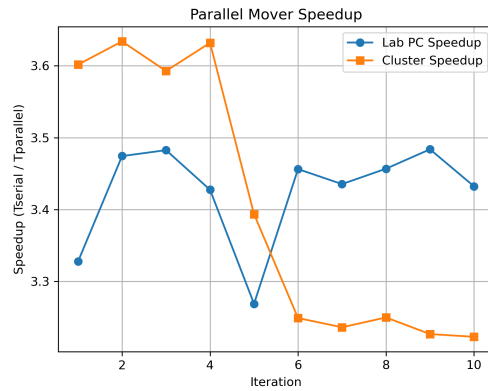


Figure 10: Speedup achieved by the parallel mover implementation.

The measured average speedups are:

- Average Lab PC Speedup = 3.42
- Average HPC Cluster Speedup = 3.40

8 Questions and Analysis

8.1 Performance Differences Between Lab PC and HPC Cluster

The observed performance differences arise from architectural characteristics. The Lab PC executes the program in a dedicated environment with minimal resource contention, whereas the HPC cluster is a shared system where multiple jobs may compete for CPU and memory bandwidth.

Additionally, differences in processor clock speeds, cache sizes, and memory hierarchy can affect execution time.

8.2 Speedup from Parallelization

The parallel mover implementation achieves significant speedup because particle updates are independent of each other. Each thread processes a subset of the particles, enabling efficient workload distribution.

The achieved speedup approaches the ideal value for four threads, although it is slightly lower due to thread management overhead and memory bandwidth limitations.

8.3 Parallelizing the Interpolation Function

Parallelizing the interpolation function is more challenging because multiple particles may update the same grid nodes simultaneously. This creates potential race conditions where concurrent threads attempt to write to the same memory location.

Possible approaches include using atomic operations or maintaining thread-local copies of the grid followed by a reduction step. However, these approaches introduce additional overhead and memory usage, which may reduce performance.

9 Conclusion

This study analyzed the performance characteristics of a particle interpolation algorithm and a particle mover operation. The experiments demonstrate that the interpolation algorithm scales linearly with particle count and is largely independent of grid resolution.

The mover operation benefits significantly from OpenMP parallelization due to the independence of particle updates. The results highlight the importance of memory access patterns, hardware architecture, and parallel computing techniques in improving the performance of particle-based simulations.